

Tarea 2 — Implementar entrega causal y asignación de perfiles

Página y archivos del trabajo

La página de Tarea 2 en el sitio del curso reúne la versión HTML de estas instrucciones y el material de apoyo:

<https://cursos-202610-estudiantes.pavel-alba.chatgpt.site/cursos/cmp5001/recorrido/recursos/tarea-2/>

El material de apoyo incluido en este PDF también está organizado en esa página:

- Datos y contratos de Tarea 2
- Plantilla opcional de README

Descarga también de esa página los archivos para trabajar:

- `CASES.json`
- `analysis.py`
- `run_cases.py`
- `check_contract.py`
- `test_student.py`

Un servicio guarda perfiles en varios nodos y envía mensajes cuando cambian. Tu programa debe decidir cuándo puede entregar cada mensaje sin adelantarse a sus dependencias y qué nodo tendrá cada perfil al añadir un servidor. Ya recibes los datos y el programa que los lee. Implementa las reglas, pruébalas con casos propios y compara dos métodos de asignación.

Entregables y formato

Entrega un solo ZIP con `analysis.py`, `test_student.py`, `run_cases.py`, `check_contract.py`, `CASES.json` y un README. Usa `README.md` o, si lo necesitas, `README.txt`, `README.docx` o `README.pdf`. No entregues capturas, otro informe ni toda la salida del programa. Usa Python 3.11+ y la biblioteca estándar; no necesitas cuentas ni conexión de red.

Implementación

Lee los datos y contratos. Completa las cuatro partes marcadas en `analysis.py`:

1. `tick`: actualiza el reloj Lamport y el vector en un evento local, envío o recepción. Al recibir, fusiona el estado remoto y después incrementa el componente local. No modifiques los argumentos de entrada.
2. `CausalInbox.receive`: acepta solo la primera copia de cada mensaje. Deja en el buffer los mensajes que esperan su turno o una dependencia. Cada vez que entregues uno, revisa el buffer de nuevo hasta que ninguno más esté listo. Devuelve los `id` entregados durante esta llamada.
3. `owner`: calcula el nodo propietario mediante módulo o el primer sucesor del ring; contempla el retorno al inicio del ring.
4. `compare`: usa `owner` para calcular los propietarios antes y después de añadir un nodo, las claves que cambian, la carga de cada nodo y los saltos de consulta. `run_cases.py` llama a `compare` por separado para la muestra inicial y la variante con una clave más solicitada.

Las firmas y el estado inicial están en `analysis.py`; las reglas y los datos, en el documento de contratos. `run_cases.py` lee `CASES.json` y muestra lo que calculan tus funciones: no contiene las respuestas. Puedes adaptar un ejemplo visto en clase, pero tus pruebas deben usar otros valores. No implementes transporte, persistencia, reenvío fiable ni migración de perfiles.

Probar y comparar

Guarda los cinco archivos descargados en una misma carpeta (por ejemplo, `student/`). Ejecuta estos comandos desde esa carpeta:

```
python3 check_contract.py
python3 run_cases.py
python3 -m unittest -v
```

Primero haz pasar los ejemplos públicos. Después reemplaza el test de ejemplo en `test_student.py` por tres pruebas ejecutables con entradas distintas de las suministradas:

- una prueba de reloj que detecte si se incrementa el vector antes de fusionarlo al recibir;
- una prueba de entrega que detecte un mensaje entregado antes de cumplir sus dependencias • entregado dos veces;
- una prueba de asignación que detecte un retorno incorrecto del ring • un cálculo incorrecto de la carga de una clave muy solicitada.

En las dos últimas pruebas, basta con detectar uno de los dos errores indicados.

Calcula el resultado esperado sin usar la función que estás probando. Una prueba que solo comprueba que el programa terminó no verifica ninguna regla.

Ejecuta la muestra inicial y la variante con más solicitudes a la clave 10. En el README incluye los comandos usados y responde brevemente:

1. Después de añadir D, ¿qué método cumple **ambos** límites en la muestra inicial: owner distinto para a lo sumo tres claves y carga máxima 16?
2. ¿Sigue cumpliéndolos cuando la frecuencia de la clave 10 pasa a 20?
3. ¿Cuántos saltos de consulta requiere cada método? ¿Por qué calcular el nuevo owner no demuestra que los perfiles ya se hayan movido?

Usa números de tus resultados para justificar la decisión. No pegues las tablas completas que imprime el programa.

Restricciones

No cambies los datos suministrados ni reemplaces las funciones por respuestas fijas. Sí puedes crear entradas propias para las pruebas. No se exigen finger tables, redes reales, replicación ni algoritmos posteriores. Si el entorno suministrado falla, conserva el código y contacta al docente: una falla del paquete no se califica como error conceptual.

Criterios de evaluación

Relojes 15; entrega causal 25; asignación y medición 40; pruebas propias 15; decisión y límite 5. **Total: 100 puntos.**

La tarea está diseñada para unas 3-4 horas de trabajo individual. La evaluación se centra en que el código siga las reglas y las pruebas detecten errores. La extensión del README no otorga puntos.

Datos y contratos de Tarea 2

CASES.json contiene entradas, no respuestas. run_cases.py las procesa con tu código e imprime los resultados. No necesitas editarlo ni copiar su salida.

Relojes

En CASES.json, los eventos c^* , a^* y b^* ocurren en C, A y B, respectivamente. Sus índices son 0, 1 y 2. En un evento local o envío, incrementa el reloj Lamport y el componente propio del vector. Al recibir, el nuevo Lamport es $\max(\text{local}, \text{remoto}) + 1$: primero toma el máximo de cada componente vectorial y después incrementa el componente del receptor. La firma de tick indica cómo se recibe el estado remoto.

La lista respeta el orden de cada proceso y de envío/recepción. No establece un orden total entre procesos. received indica el evento que envió el mensaje, o null si no hubo recepción. Las siguientes medidas son solo ejemplos para interpretar la historia: no son campos de CASES.json ni debes calcularlas. Aunque la hora de pared marque $c3=10:00:00.020$ y $b1=09:59:59.900$, el mensaje establece $c3 \rightarrow b1$. La duración monotónica de $c1 \rightarrow c6$ dentro de C es 160 ms; no restes relojes de máquinas distintas.

Admisión y entrega

id identifica el mensaje: todas sus copias tienen el mismo id. sender es 0, 1 o 2; seq empieza en 0 para cada emisor. Acepta solo la primera copia. Si llega otra con el mismo id, no cambies seen ni delivered. Las llegadas ya están dadas; no implementes reenvíos ni fiabilidad de red.

El vector de tick y el contador delivered pertenecen a entradas distintas. delivered[p] cuenta los mensajes ya entregados del proceso p. deps[p] indica cuántos mensajes de p deben haberse entregado antes de este. Entrega un mensaje solo si $\text{seq} == \text{delivered}[\text{sender}]$ y, para todo p, $\text{deps}[p] \leq \text{delivered}[p]$. Después incrementa delivered[sender].

Revisa el buffer hasta que no haya más mensajes listos. En cada recorrido, empieza por el que llegó primero. Si dos mensajes independientes están listos, este orden solo resuelve el empate: no crea una relación causal. Si falta una dependencia, deja el mensaje en el buffer.

Asignación y medida

frequencies[i] es el número de solicitudes para keys[i]. Las claves enteras ya son los valores de hash, entre 0 y 15: no calcules otro hash. En modo modulo, usa el orden de inserción de los nodos y $\text{key} \% \text{número_de_nodos}$. En modo ring, el owner es el primer nodo cuya posición es $\geq \text{key}$; si no existe, vuelve al nodo de menor posición.

compare recibe frequencies como diccionario clave→solicitudes y devuelve:

- before y after: diccionarios clave→owner;
- moved: claves cuyo owner cambia, en el orden de keys;
- load_before y load_after: solicitudes por nodo, incluidos los nodos con cero;
- hops: saltos totales de la **petición de consulta**, una vez por clave, después del cambio. No cuenta la respuesta del propietario al cliente.

En modulo, el cliente calcula el owner y le envía la consulta directamente: un salto por clave. En ring, el cliente no tiene el mapa: envía la consulta al nodo de menor posición (A en la muestra), lo que cuesta un salto. Cada avance al siguiente nodo suma otro. Si el owner es A, la consulta termina allí en un salto, incluso si A es el sucesor por retorno circular. No multipliques los saltos por la frecuencia: esta solo sirve para calcular la carga.

Compara la asignación antes y después de añadir D. El método elegido debe cambiar el owner de a lo sumo tres claves y dejar una carga máxima de 16. Después cambia solo f(10) a 20 y compara otra vez. Cada clave representa un perfil: compare calcula su nuevo owner, pero no mueve el perfil ni coordina cambios entre nodos.

Plantilla opcional de README

Ejecutar

Escribe los comandos que usaste.

Resultado y decisión

Con D añadido, indica qué método cumple los dos límites: owner distinto para hasta tres claves y carga máxima 16. ¿Cambia la decisión con $f(10)=20$? Usa los números obtenidos y compara los saltos de consulta.

Límite

Explica por qué calcular un nuevo owner no demuestra que los perfiles ya se hayan movido.

No pegues las tablas completas ni adjuntes otro informe.